

OpenBox Policy Studio: beginner's guide

Code and setup: <https://github.com/OpenBox-AI/openbox-policy-studio>

Upload a client's NDA and the Policy Studio suggests the OpenBox rules that keep your agent in line with it: which of the client's documents the agent may read or file, whether it needs approval first, and whether it may send anything outside. You check each rule, tick the ones you agree with, and apply them. Nothing changes in OpenBox until you press Apply.

If you have never used OpenBox or the Studio, read this page from the top. It takes about 15 minutes to set up the first client.

Words you will see

Word	What it means
OpenBox	The platform that watches your agent's actions and stops, holds or allows each one according to rules. You use it through its dashboard.
Agent	The software assistant whose actions OpenBox governs, for example one that reads a client's documents and writes briefings. It must already be registered in OpenBox.
Tool	One action the agent can take, such as <code>read_document</code> , <code>upload_document</code> or <code>send_email</code> . Rules apply to tools.
Policy rule	One instruction to OpenBox, for example "block <code>read_document</code> for anything in folder <code>0005/70001/</code> ".
Firm	A client whose NDA you are applying. Each firm has its own binding.
Binding	A small settings file per firm that tells the Studio which agent the rules go on, which folder holds that client's documents , and what the client is called. Without a binding the Studio does not know where to put the rules.

How the Studio knows which agent to protect: the binding

An NDA talks about "the Disclosing Party", "Representatives" and "Confidential Information". It never names your agent or your folders. The binding is where you fill that gap, once per client:

- **Which agent** gets the rules: its OpenBox agent ID.
- **Which folder** holds this client's documents, as the agent sees it. The rules block, hold or sandbox the agent's reads and filings under that folder.
- **What the client is called**, including other names and project codenames, so the Studio can recognise them in the NDA.

Every binding is a file in the `bindings/` folder, named after the firm, for example `bindings/atlas.yaml`. Every file there appears in the Studio's **Firm** menu.

Step 1: find your agent in OpenBox

1. Sign in to the OpenBox dashboard and open **Agents**.
2. If your agent is not there yet, ask whoever runs the agent to register it, or register it from the binding (see "Check or register the agent" below).
3. Click the agent. Its **agent ID** is the long code in the address bar after `/agents/`, for example `16e4c8e5-0921-4946-ae5b-96fe1966e57d`.

The agent must be in the same OpenBox organisation as the API key you put in `.env` (see Setup below), or the Studio cannot see it or add rules to it.

Step 2: create the binding

The easy way, in the Studio. Choose **Create firm from a contract** under the Firm menu and upload the client's contract or NDA. The Studio fills in the client's names, codenames and purpose for you. Then check two fields yourself, because the contract cannot tell the Studio either of them:

- **Agent these policies apply to:** pick your agent from the list. The list shows the agents in your OpenBox organisation.
- **Folder holding the firm's material:** where this client's documents live, written the way the agent sees it, for example `0005/70001/`.

Click **Create firm and compile**. The Studio writes `bindings/<firm>.yaml` and the firm appears in the Firm menu.

By hand. Copy [docs/binding-template.yaml](#) into `bindings/`, rename it after the client (for example `bindings/northwind.yaml`) and fill it in. A complete one looks like this:

```
firm: atlas                                # short name; also the file name
disclosing_party: Atlas Grid Energy plc
disclosing_party_aliases:                  # other names the NDA uses for the client
- Atlas Grid Energy
- Atlas
receiving_party: Sellist Advisory LLP
codenames:                                # project names and markings to keep out of the
agent's writing
- Project Halcyon
purpose: advising on the proposed acquisition of the Portfolio
covered_folders:                          # where this client's documents live, as the agent
sees them
- 0005/70001/
agent:                                     # THE AGENT THE RULES GO ON
  id: 16e4c8e5-0921-4946-ae5b-96fe1966e57d # from the OpenBox dashboard (Step 1)
  name: compliance agent                     # any label; shown in the Studio
```

Check or register the agent

Once the binding exists, one command confirms its agent is real:

```
uv run python scripts/bootstrap_agents.py bindings/northwind.yaml
```

- If `agent.id` is filled in, it checks that agent exists in the organisation of your `OPENBOX_ORG_API_KEY`, and tells you if it does not. A wrong ID is the most common reason rules end up somewhere unexpected.
- If `agent.id` is still the template's placeholder (`00000000-0000-0000-0000-000000000000`), it registers a new agent in OpenBox under `agent.name`, writes the new ID into the binding, and saves the agent's own keys to `.env.agents` for whoever runs the agent.

Step 3: tell the Studio your agent's tools

Rules are written against the agent's tool names. If a name is wrong, the rule exists in OpenBox but never matches anything the agent does. There are two ways to tell the Studio the tools:

List them in the binding (simplest). Add these lines to the firm's file, using your agent's real tool names:

```
read_tools:          # tools that open a document by its ID or path
- read_document
file_tools:          # tools that save or upload into a client folder
- upload_document
outbound_tools:      # tools that send anything outside: email, HTTP, chat
- send_email
- http_post
```

If you leave these out, the Studio assumes exactly the names above. It also assumes the read tool takes the document's location in an argument called `document_id`, and the filing tool in `destination_document_id`. If your agent's tools use other argument names, use the next option instead.

Export the agent's graph (most accurate). For an agent built with LangGraph, a developer can export its tools, argument names included, to `graphs/<agent id>.json`; [scripts/export_graph.py](#) shows how. When that file exists, the Studio reads the tools from it and ignores the lists above.

Step 4: check it worked

After you apply rules, click **Open in OpenBox**. You should land on your agent's **Policies** tab and see one rule per proposed line, named after the firm and the clause, for example `NDA atlas §2 read_document 0005/70001/`. If the rules are on a different agent, the binding's `agent.id` is wrong. Fix it and apply again.

Setup

You need [uv](#) (a Python tool runner) and the keys below.

```
git clone https://github.com/OpenBox-AI/openbox-policy-studio.git
cd openbox-policy-studio
uv sync
cp .env.example .env      # then open .env and fill in the keys
uv run nda-web            # then open http://127.0.0.1:8010
```

Keys

Key	Needed for	Where to get it
OPENBOX_ORG_API_KEY	listing your agents and adding rules to them	OpenBox dashboard, your organisation's settings (starts with <code>obx_key_</code>)
OPENBOX_BACKEND_URL	which OpenBox to talk to	<code>https://api.openbox.ai</code> , or your environment's API address
ANTHROPIC_API_KEY	reading each clause of the NDA (Claude Sonnet 5.5)	console.anthropic.com
OPENAI_API_KEY	sorting clauses and checking rules (OpenAI Decisions API, <code>gpt-6-luna</code>)	platform.openai.com
TYPESAFE_API_KEY	optional alternative to the OpenAI key for sorting and checking	TypeSafe

Leave a key blank and that step runs in a built-in offline mode, so you can try the Studio before you have every key; the rules it suggests will be much less accurate. Without `OPENBOX_ORG_API_KEY`, Apply only shows what it would send.

The OpenBox environment must offer the policy-rule API (`/agent/:id/policy-rule`); the Studio adds rules through it.

Using it

1. Pick the client from the **Firm** menu (or create one, Step 2 above).
2. Click **Upload an NDA** and choose the file: PDF, Word or Markdown. In about ten seconds you see the **Proposed policy rules**, each beside the clause it came from.
3. Read each rule against its clause. Rules marked **Needs review** are the ones to look at first.
4. Tick the rules you agree with and click **Apply to OpenBox**. Each rule is tested with a sample call it must stop and one it must allow, then switched on. Uploading the same NDA again later is safe: rules already in place are left alone and changed ones are updated, not duplicated.

What each rule does to the agent:

Decision	Effect
BLOCK	the action is refused
REQUIRE_APPROVAL	the action waits until someone approves it in OpenBox
HALT	the action is refused and the agent's whole run stops
Sandbox (CONSTRAIN)	the action may run, but only inside the sandbox

Some clauses never become rules, such as how long the NDA lasts, returning documents, or governing law. They cannot be checked on the agent's actions, and the Studio lists them rather than dropping them silently.

From the command line, the same steps without the web page:

```
uv run nda-compile path/to/nda.pdf --bindings bindings/atlas.yaml
```